WebScraping_BeautifulSoup_Tutorial (1).ipynb

Cell 1 (markdown)

Web Scraping with Python & BeautifulSoup

In this notebook you will learn:
- How HTTP requests work under the hood
- How to fetch web pages with the `requests` library
- How to parse HTML with `BeautifulSoup`
- How to extract text, links, attributes, and tables
- How to scrape multiple pages (pagination)
- How to save data to CSV and JSON
- Ethical scraping practices

> **Practice site used:** quotes.toscrape.com and books.toscrape.com — both are built specifically for scraping practice, so it is completely legal and welcome.

Cell 2 (markdown)

Section 1 — Installation & Imports

First, let's install the two core libraries we need.

Cell 3 (code)

```
# Run this cell once to install dependencies
!pip install requests beautifulsoup4
```

Cell 4 (code)

```
# Core imports we'll use throughout this notebook
import requests
from bs4 import BeautifulSoup
import time
import csv
import json
import os

print("■ All libraries imported successfully!")
```

Cell 5 (markdown)

Section 2 — How the Web Works

Before scraping, it helps to understand what actually happens when you visit a URL.

```
You (script) ■■■■ GET https://example.com ■■■■■ Web Server
■■■■ 200 OK + HTML body ■■■■■■■■■■■ Web Server
```

```

1. **You send an HTTP GET request** to a URL
2. **The server replies** with a status code and the page's HTML
3. **You parse the HTML** to extract the data you want

### HTTP Status Codes
| Code | Meaning |
|---|---|
| 200 | ■ OK — everything worked |
| 301/302 | ■■ Redirect |
| 403 | ■ Forbidden — server refuses |
| 404 | ■ Not Found |
| 429 | ■ Too Many Requests — slow down! |
| 500 | ■ Server Error |

## *Cell 6 (code)*

```
# Let's make our first HTTP request
url = "https://quotes.toscrape.com"

response = requests.get(url)

print(f"Status Code : {response.status_code}")
print(f"Content-Type: {response.headers.get('Content-Type')}")
print(f"\nFirst 500 characters of HTML:")
print("-" * 50)
print(response.text[:500])
```

## *Cell 7 (markdown)*

### ■ Setting Headers — Look Like a Browser

Some websites block requests that don't come from a real browser.
Always set a `User-Agent` header as good practice.

## *Cell 8 (code)*

```
# Set a User-Agent header to identify your scraper
HEADERS = {
"User-Agent": "Mozilla/5.0 (compatible; LearnerBot/1.0; Educational project)"
}

response = requests.get(url, headers=HEADERS)
print(f"Status: {response.status_code}")
print("Headers sent successfully ■")
```

Cell 9 (markdown)

Handling Errors Properly

Cell 10 (code)

```
# Always handle potential errors
def safe_get(url, headers=HEADERS):
```
```

```
"""Fetch a URL and return the response, or None on error."""
try:
r = requests.get(url, headers=headers, timeout=10)
r.raise_for_status() # Raises an exception for 4xx/5xx codes
return r
except requests.HTTPError as e:
print(f"HTTP error: {e}")
except requests.ConnectionError:
print("Connection error — check your internet.")
except requests.Timeout:
print("Request timed out.")
return None

# Test with a good URL
r = safe_get("https://quotes.toscrape.com")
print("Good URL:", r.status_code if r else "Failed")

# Test with a bad URL
r = safe_get("https://quotes.toscrape.com/this-page-does-not-exist")
print("Bad URL result:", r)
```

Cell 11 (markdown)

```
---
## Section 3 — BeautifulSoup Basics

`BeautifulSoup` turns raw HTML text into a **navigable tree structure** you can search and extract from.

```python
from bs4 import BeautifulSoup

html = "<h1 class='title'>Hello</h1><p>World</p>"
soup = BeautifulSoup(html, 'html.parser')
```

The Parsing Tree
```
html
■■■ body
■■■ h1.title → "Hello"
■■■ p → "World"
```
```

Cell 12 (code)

```
# Create a soup object from a page
response = requests.get("https://quotes.toscrape.com", headers=HEADERS)
soup = BeautifulSoup(response.text, "html.parser")

# Inspect the page title
print("Page title:", soup.title.text)
print("\nType of soup:", type(soup))
```

Cell 13 (markdown)

■ `find()` vs `find_all()`

Method	Returns	Use when
`find(tag)`	First match (or `None`)	You need just one element
`find_all(tag)`	List of all matches	You need all matching elements
`select_one('css')`	First CSS match	CSS selectors (powerful!)
`select('css')`	List of CSS matches	CSS selectors for all

Cell 14 (code)

```
# ■■ find(): returns the FIRST match ■■■■■■■■■■■■■■■■■■■■■■■
first_quote = soup.find("span", class_="text")
print("First quote:")
print(first_quote.get_text())

print()

# ■■ find_all(): returns a LIST of all matches ■■■■■■■■■■■■■■
all_quotes = soup.find_all("span", class_="text")
print(f"Total quotes found: {len(all_quotes)}")
print()

# Print each quote
for i, q in enumerate(all_quotes, 1):
print(f"{i}. {q.get_text()}...")
```

Cell 15 (code)

```
# ■■ CSS Selectors with select() ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# CSS selectors are more powerful and flexible

# Select by class name: ".classname"
authors = soup.select(".author")
print("Authors:")
for a in authors:
print(f" - {a.text}")
```

Cell 16 (markdown)

```
---
## Section 4 — Extracting Different Types of Data

### Getting Text, Attributes, and Links
```

Cell 17 (code)

```
# ■■ Extracting text ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
element = soup.find("h1")
print(".text :", element.text)
print(".get_text() :", element.get_text())
print(".get_text(strip=True):", element.get_text(strip=True)) # removes whitespace
```

Cell 18 (code)

```
# ■■ Extracting attributes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# HTML: <a href="/author/Albert-Einstein">Albert Einstein</a>

all_links = soup.find_all("a")

print("All links on the page:")
for link in all_links[:10]:
href = link.get("href") # .get() returns None if attribute missing
text = link.get_text(strip=True)
print(f" {text:30s} → {href}")
```

Cell 19 (code)

```
# ■■ Navigating the tree: parent, children, siblings ■■■■■■
quote_div = soup.find("div", class_="quote")

print("Quote div contents:")
print(quote_div.prettify()[:400])
```

Cell 20 (code)

```
# Access children directly
print("Quote text :", quote_div.find("span", class_="text").get_text())
print("Author :", quote_div.find("small", class_="author").get_text())
print("Tags :", [t.text for t in quote_div.select(".tag")])
```

Cell 21 (markdown)

```
---
## Section 5 — Full Page Scrape: Quotes

Let's put it all together and extract structured data from a complete page.
```

Cell 22 (code)

```
def scrape_quotes_page(url):
"""Scrape all quotes from a single page. Returns a list of dicts."""
response = safe_get(url)
if not response:
return []

soup = BeautifulSoup(response.text, "html.parser")
results = []

for div in soup.select(".quote"):
# Use select_one for single element, with fallback
text_el = div.select_one(".text")
author_el = div.select_one(".author")
tag_els = div.select(".tag")

if not text_el or not author_el:
continue # Skip malformed entries

results.append({
"text" : text_el.get_text(strip=True),
"author": author_el.get_text(strip=True),
```

```
"tags" : [t.get_text() for t in tag_els],
})

return results

# Test on page 1
quotes = scrape_quotes_page("https://quotes.toscrape.com/page/1/")
print(f"Scraped {len(quotes)} quotes\n")
for q in quotes[:10]:
print(f" Author : {q['author']}")
print(f" Quote : {q['text'][:70]}...")
print(f" Tags : {q['tags']}")
print()
```

Cell 23 (markdown)

```
---
## Section 6 — Scraping Multiple Pages (Pagination)

Most sites split their data across multiple pages. The pattern is:

1. Scrape the current page
2. Find the "next page" link
3. Follow it and repeat
4. Stop when there's no "next" link
```

Cell 24 (code)

```
def get_next_page_url(soup, base_url="https://quotes.toscrape.com"):
"""Find the 'Next' button URL, or return None if we're on the last page."""
next_li = soup.find("li", class_="next")
if next_li:
#Get Reference for Next Page like: /page2/
next_href = next_li.find("a").get("href")
#Merge with original url so it becomes: https://quotes.toscrape.com/page2/
return base_url + next_href
#If no next button
return None

# Test it
response = requests.get("https://quotes.toscrape.com/page/1/", headers=HEADERS)
soup = BeautifulSoup(response.text, "html.parser")
print("Next page URL:", get_next_page_url(soup))

response2 = requests.get("https://quotes.toscrape.com/page/10/", headers=HEADERS)
soup2 = BeautifulSoup(response2.text, "html.parser")
print("Last page next:", get_next_page_url(soup2))
```

Cell 25 (code)

```
# ■■ Scrape ALL pages (follow next links) ■■■■■■■■■■■■■■■■■■
def scrape_all_quotes(start_url="https://quotes.toscrape.com", max_pages=5):
"""
Scrape quotes across multiple pages.
max_pages: safety limit — remove or increase for full site.
```

```
"""
all_quotes = []
current_url = start_url
page_num = 1

while current_url and page_num <= max_pages:
print(f" Scraping page {page_num}: {current_url}")

response = safe_get(current_url)
if not response:
break

soup = BeautifulSoup(response.text, "html.parser")

# Scrape this page
page_quotes = scrape_quotes_page(current_url)
all_quotes.extend(page_quotes)
print(f" ✓ Got {len(page_quotes)} quotes (total: {len(all_quotes)})")

# Find next page
current_url = get_next_page_url(soup)
page_num += 1

time.sleep(1) # ■■ Rate limit: wait 1 second between pages

print(f"\nDone! Collected {len(all_quotes)} quotes total.")
return all_quotes

all_quotes = scrape_all_quotes(max_pages=5)
```

Cell 26 (markdown)

```
---
## Section 7 — Scraping a Product Listing (books.toscrape.com)

Let's tackle a more complex page with product cards, prices, and ratings.
```

Cell 27 (code)

```
def scrape_books(url="https://books.toscrape.com"):
"""Scrape book titles, prices, ratings, and availability."""
response = safe_get(url)
if not response:
return []

soup = BeautifulSoup(response.text, "html.parser")
books = []

# Rating words to numbers
rating_map = {"One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5}

for article in soup.select("article.product_pod"):
title_tag = article.find("h3").find("a")
price_tag = article.select_one(".price_color")
rating_tag = article.find("p", class_="star-rating")
```

```
avail_tag = article.select_one(".availability")

title = title_tag.get("title") if title_tag else "N/A"
price = price_tag.get_text(strip=True) if price_tag else "N/A"
rating_word = rating_tag["class"][1] if rating_tag else "Zero"
rating = rating_map.get(rating_word, 0)
avail = avail_tag.get_text(strip=True) if avail_tag else "N/A"

books.append({
"title" : title,
"price" : price,
"rating_stars": rating,
"availability": avail,
})

return books

books = scrape_books()
print(f"Scraped {len(books)} books\n")
print(f"{'Title':<45} {'Price':>8} {'■':>4} Availability")
print("-" * 72)
for b in books[:10]:
print(f"{b['title'][:44]:<45} {b['price']:>8} {b['rating_stars']:>4} {b['availability']}")
```

Cell 28 (markdown)

■ Section 8 — Saving Your Data

After scraping, you'll usually want to save data to a file for later use.

Cell 29 (code)

```
# ■■ Save to CSV ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def save_to_csv(data, filename, fieldnames=None):
"""Save a list of dicts to a CSV file."""
if not data:
print("No data to save.")
return

if fieldnames is None:
fieldnames = list(data[0].keys())

with open(filename, "w", newline="", encoding="utf-8") as f:
writer = csv.DictWriter(f, fieldnames=fieldnames)
writer.writeheader()
writer.writerows(data)

print(f"■ Saved {len(data)} rows to '{filename}'")

save_to_csv(all_quotes, "quotes.csv")
save_to_csv(books, "books.csv")
```

Cell 30 (code)

```
# ■■ Save to JSON ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def save_to_json(data, filename):
"""Save a list of dicts to a JSON file."""
with open(filename, "w", encoding="utf-8") as f:
json.dump(data, f, indent=2, ensure_ascii=False)
print(f"■ Saved {len(data)} records to '{filename}'")

save_to_json(all_quotes, "quotes.json")

# Preview JSON output
with open("quotes.json") as f:
preview = json.load(f)
print("\nJSON preview (first entry):")
print(json.dumps(preview[0], indent=2))
```

Cell 31 (code)

```
# ■■ Load and verify saved CSV ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import csv

with open("quotes.csv", encoding="utf-8") as f:
reader = csv.DictReader(f)
rows = list(reader)

print(f"CSV loaded: {len(rows)} rows")
print("Columns:", list(rows[0].keys()))
print("\nFirst row:")
for k, v in rows[0].items():
print(f" {k}: {v}")
```

Cell 32 (markdown)

■ Section 9 — Caching HTML During Development

While writing your parser, you'll run your code many times.
Caching saves the HTML to disk so you don't re-fetch the page on every run — this is faster, kinder to servers, and works offline.

Cell 33 (code)

```
def fetch_with_cache(url, cache_dir="html_cache"):
"""
Fetch a URL, caching the HTML to disk.
On subsequent calls with the same URL, loads from cache.
"""
os.makedirs(cache_dir, exist_ok=True)

# Create a safe filename from the URL
safe_name = url.replace("https://", "").replace("http://", "")
safe_name = safe_name.replace("/", "_").replace("?", "_") + ".html"
cache_path = os.path.join(cache_dir, safe_name)

if os.path.exists(cache_path):
print(f"[CACHE] Loading: {cache_path}")
with open(cache_path, encoding="utf-8") as f:
```

```
return f.read()

print(f"[FETCH] Downloading: {url}")
r = requests.get(url, headers=HEADERS, timeout=10)
r.raise_for_status()

with open(cache_path, "w", encoding="utf-8") as f:
f.write(r.text)

return r.text

# First call → downloads
html1 = fetch_with_cache("https://quotes.toscrape.com/page/1/")
# Second call → loads from disk (instant!)
html2 = fetch_with_cache("https://quotes.toscrape.com/page/1/")

soup = BeautifulSoup(html2, "html.parser")
print("\nPage title:", soup.find("h1").text)
```

Cell 34 (markdown)

```
---
## ■■ Section 10 — Ethics & Best Practices

Web scraping exists in a legal and ethical gray area. Following these rules keeps you safe and respectful.

### The Golden Rules

| Rule | Why |
|------|-----|
| ■ Check `robots.txt` | Sites declare what crawlers may access |
| ■ Rate limit your requests | Don't overload servers (`time.sleep`) |
| ■ Read the Terms of Service | ToS violations can have legal consequences |
| ■ Prefer official APIs | Faster, reliable, and explicitly allowed |
| ■ Cache during development | Avoid repeat requests while writing code |
| ■ Identify your bot | Use a descriptive User-Agent |
| ■ Don't scrape personal data | Privacy laws (GDPR, etc.) apply |
| ■ Don't bypass login walls | Unauthorized access laws apply |
```

Cell 35 (code)

```
# ■■ Checking robots.txt programmatically ■■■■■■■■■■■■■■■■■■
import urllib.robotparser

def can_scrape(base_url, path, user_agent="*"):
"""Check robots.txt to see if scraping a path is allowed."""
rp = urllib.robotparser.RobotFileParser()
rp.set_url(base_url.rstrip("/") + "/robots.txt")
try:
rp.read()
except Exception as e:
print(f"Could not read robots.txt: {e}")
return True # Assume OK if robots.txt is unavailable
```

```
full_url = base_url.rstrip("/") + "/" + path.lstrip("/")
allowed = rp.can_fetch(user_agent, full_url)
delay = rp.crawl_delay(user_agent)

status = "■ Allowed" if allowed else "■ Disallowed"
print(f"{status}: {full_url}")
if delay:
print(f" Requested crawl delay: {delay}s")

return allowed

can_scrape("https://quotes.toscrape.com", "/page/1/")
can_scrape("https://quotes.toscrape.com", "/login")
```

Cell 36 (code)

```
# ■■ Rate limiting helper ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import random

def polite_get(url, min_delay=1.0, max_delay=3.0):
"""
Fetch a URL with a random delay before each request.
Randomizing the delay makes scraping look more human-like.
"""
delay = random.uniform(min_delay, max_delay)
print(f" Waiting {delay:.1f}s before fetching...")
time.sleep(delay)
return safe_get(url)

# Example: scrape 3 pages politely
for page in range(1, 4):
url = f"https://quotes.toscrape.com/page/{page}/"
r = polite_get(url)
if r:
s = BeautifulSoup(r.text, "html.parser")
count = len(s.select(".quote"))
print(f" Page {page}: {count} quotes scraped ✓")
```

Cell 37 (markdown)

```
---
## ■ Section 11 — Complete Mini Project

Let's combine everything into a clean, production-ready scraper class.
```

Cell 38 (code)

```
class QuoteScraper:
"""
A complete, polite scraper for quotes.toscrape.com.

Features:
- Respects robots.txt
- Rate limiting between requests
- Error handling
```

```
- HTML caching
- Export to CSV and JSON
"""

BASE_URL = "https://quotes.toscrape.com"

def __init__(self, max_pages=5, delay=1.0, use_cache=True):
self.max_pages = max_pages
self.delay = delay
self.use_cache = use_cache
self.headers = {"User-Agent": "LearnerBot/1.0 (Educational)"}
self.quotes = []

def _get(self, url):
"""Fetch a URL with optional caching and rate limiting."""
if self.use_cache:
html = fetch_with_cache(url)
return BeautifulSoup(html, "html.parser")

time.sleep(self.delay)
r = safe_get(url, headers=self.headers)
if not r:
return None
return BeautifulSoup(r.text, "html.parser")

def _parse_page(self, soup):
"""Extract all quotes from a BeautifulSoup object."""
results = []
for div in soup.select(".quote"):
text_el = div.select_one(".text")
author_el = div.select_one(".author")
link_el = div.select_one("a")
tag_els = div.select(".tag")

if not text_el:
continue

results.append({
"text" : text_el.get_text(strip=True),
"author" : author_el.get_text(strip=True) if author_el else "",
"author_url" : self.BASE_URL + link_el.get("href") if link_el else "",
"tags" : [t.get_text() for t in tag_els],
})
return results

def scrape(self):
"""Scrape all pages up to max_pages. Returns self for chaining."""
url = self.BASE_URL

for page in range(1, self.max_pages + 1):
print(f"[Page {page}/{self.max_pages}] {url}")
soup = self._get(url)

if not soup:
print(" Failed — stopping.")
break
```

```
page_quotes = self._parse_page(soup)
self.quotes.extend(page_quotes)
print(f" ✓ {len(page_quotes)} quotes (total: {len(self.quotes)})")

# Find next page
next_li = soup.find("li", class_="next")
if not next_li:
print(" Reached last page.")
break
url = self.BASE_URL + next_li.find("a").get("href")

return self

def to_csv(self, filename="quotes_final.csv"):
save_to_csv(self.quotes, filename)
return self

def to_json(self, filename="quotes_final.json"):
save_to_json(self.quotes, filename)
return self

def top_authors(self, n=5):
"""Return the n most quoted authors."""
from collections import Counter
counts = Counter(q["author"] for q in self.quotes)
return counts.most_common(n)

def quotes_by_tag(self, tag):
"""Return all quotes that include a given tag."""
return [q for q in self.quotes if tag.lower() in [t.lower() for t in q["tags"]]]


# ■■ Run the scraper ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
scraper = QuoteScraper(max_pages=5, use_cache=True)
scraper.scrape()

print(f"\n■ Results:")
print(f" Total quotes : {len(scraper.quotes)}")
print(f" Top authors : {scraper.top_authors()}")
print(f" 'life' quotes: {len(scraper.quotes_by_tag('life'))}")
```

Cell 39 (code)

```
# Export results
scraper.to_csv("quotes_final.csv").to_json("quotes_final.json")

print("\n■ Files created:")
for f in ["quotes_final.csv", "quotes_final.json"]:
size = os.path.getsize(f)
print(f" {f} ({size:,} bytes)")
```

Cell 40 (markdown)

■ Summary — What You've Learned

Topic	Key Concepts
HTTP + requests	GET requests, status codes, headers, error handling
BeautifulSoup	`find()`, `find_all()`, `select()`, `.text`, `.get()`
Pagination	Following next-page links in a loop
Real data	Quotes, authors, tags, prices, ratings
Saving data	CSV (`csv.DictWriter`) and JSON (`json.dump`)
Caching	Saving HTML locally during development
Ethics	`robots.txt`, rate limits, ToS, User-Agent
OOP design	Wrapping it all in a clean `Scraper` class

■ Next Steps

- **`scrapy`** — industrial-strength scraping framework with pipelines
- **`selenium`** / **`playwright`** — scraping JavaScript-rendered pages
- **`aiohttp`** — async scraping for high-speed parallel requests
- **Databases** — store scraped data in SQLite, PostgreSQL, or MongoDB
- **Proxy rotation** — avoid IP bans on large-scale scraping
- **Cloud scraping** — run scrapers on AWS Lambda or GitHub Actions

---
Happy scraping! Remember: be polite, be ethical, and always check `robots.txt`. ■